

# breastcancer-analysis

October 7, 2026

## 1 Decision Trees and Cross Validation

### 1.1 Introduction

This analysis investigates the use of decision tree classification to predict whether a breast tumour is malignant or benign using measurements of cell nuclei obtained from images of breast masses. The data set contains characteristics of the cell nuclei, including measurements relating to their radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension. For each characteristic, measurements are provided for the mean value, the standard error, and the worst observed values.

The aim of this analysis is to investigate whether these three groups of measurements differ in their ability to predict tumour diagnosis. Three separate decision tree classifiers are created: one using the mean characteristics, one using the standard error characteristics, and one using the worst characteristics.

### 1.2 Exploratory Data Analysis (EDA)

Before constructing the models, EDA is used to examine the structure and quality of the data. This includes checking the number of variables, identifying missing values, and removing variables that do not provide useful information. The cleaned data are then separated into the three groups of measurements for classification.

```
[113]: #Importing the relevant libraries and loading the data set
import pandas as pd
import numpy as np
df = pd.read_csv("data.csv")
df
```

```
[113]:
```

|     | id       | diagnosis | radius_mean | texture_mean | perimeter_mean | area_mean | \ |
|-----|----------|-----------|-------------|--------------|----------------|-----------|---|
| 0   | 842302   | M         | 17.99       | 10.38        | 122.80         | 1001.0    |   |
| 1   | 842517   | M         | 20.57       | 17.77        | 132.90         | 1326.0    |   |
| 2   | 84300903 | M         | 19.69       | 21.25        | 130.00         | 1203.0    |   |
| 3   | 84348301 | M         | 11.42       | 20.38        | 77.58          | 386.1     |   |
| 4   | 84358402 | M         | 20.29       | 14.34        | 135.10         | 1297.0    |   |
| ..  | ...      | ...       | ...         | ...          | ...            | ...       |   |
| 564 | 926424   | M         | 21.56       | 22.39        | 142.00         | 1479.0    |   |
| 565 | 926682   | M         | 20.13       | 28.25        | 131.20         | 1261.0    |   |
| 566 | 926954   | M         | 16.60       | 28.08        | 108.30         | 858.1     |   |

|     |        |   |       |       |        |        |
|-----|--------|---|-------|-------|--------|--------|
| 567 | 927241 | M | 20.60 | 29.33 | 140.10 | 1265.0 |
| 568 | 92751  | B | 7.76  | 24.54 | 47.92  | 181.0  |

|     | smoothness_mean | compactness_mean | concavity_mean | concave points_mean | \ |
|-----|-----------------|------------------|----------------|---------------------|---|
| 0   | 0.11840         | 0.27760          | 0.30010        | 0.14710             |   |
| 1   | 0.08474         | 0.07864          | 0.08690        | 0.07017             |   |
| 2   | 0.10960         | 0.15990          | 0.19740        | 0.12790             |   |
| 3   | 0.14250         | 0.28390          | 0.24140        | 0.10520             |   |
| 4   | 0.10030         | 0.13280          | 0.19800        | 0.10430             |   |
| ..  | ...             | ...              | ...            | ...                 |   |
| 564 | 0.11100         | 0.11590          | 0.24390        | 0.13890             |   |
| 565 | 0.09780         | 0.10340          | 0.14400        | 0.09791             |   |
| 566 | 0.08455         | 0.10230          | 0.09251        | 0.05302             |   |
| 567 | 0.11780         | 0.27700          | 0.35140        | 0.15200             |   |
| 568 | 0.05263         | 0.04362          | 0.00000        | 0.00000             |   |

|     | texture_worst | perimeter_worst | area_worst | smoothness_worst | \ |
|-----|---------------|-----------------|------------|------------------|---|
| 0   | 17.33         | 184.60          | 2019.0     | 0.16220          |   |
| 1   | 23.41         | 158.80          | 1956.0     | 0.12380          |   |
| 2   | 25.53         | 152.50          | 1709.0     | 0.14440          |   |
| 3   | 26.50         | 98.87           | 567.7      | 0.20980          |   |
| 4   | 16.67         | 152.20          | 1575.0     | 0.13740          |   |
| ..  | ...           | ...             | ...        | ...              |   |
| 564 | 26.40         | 166.10          | 2027.0     | 0.14100          |   |
| 565 | 38.25         | 155.00          | 1731.0     | 0.11660          |   |
| 566 | 34.12         | 126.70          | 1124.0     | 0.11390          |   |
| 567 | 39.42         | 184.60          | 1821.0     | 0.16500          |   |
| 568 | 30.37         | 59.16           | 268.6      | 0.08996          |   |

|     | compactness_worst | concavity_worst | concave points_worst | symmetry_worst | \ |
|-----|-------------------|-----------------|----------------------|----------------|---|
| 0   | 0.66560           | 0.7119          | 0.2654               | 0.4601         |   |
| 1   | 0.18660           | 0.2416          | 0.1860               | 0.2750         |   |
| 2   | 0.42450           | 0.4504          | 0.2430               | 0.3613         |   |
| 3   | 0.86630           | 0.6869          | 0.2575               | 0.6638         |   |
| 4   | 0.20500           | 0.4000          | 0.1625               | 0.2364         |   |
| ..  | ...               | ...             | ...                  | ...            |   |
| 564 | 0.21130           | 0.4107          | 0.2216               | 0.2060         |   |
| 565 | 0.19220           | 0.3215          | 0.1628               | 0.2572         |   |
| 566 | 0.30940           | 0.3403          | 0.1418               | 0.2218         |   |
| 567 | 0.86810           | 0.9387          | 0.2650               | 0.4087         |   |
| 568 | 0.06444           | 0.0000          | 0.0000               | 0.2871         |   |

|   | fractal_dimension_worst | Unnamed: 32 |
|---|-------------------------|-------------|
| 0 | 0.11890                 | NaN         |
| 1 | 0.08902                 | NaN         |
| 2 | 0.08758                 | NaN         |
| 3 | 0.17300                 | NaN         |

```

4          0.07678      NaN
..          ...      ...
564        0.07115      NaN
565        0.06637      NaN
566        0.07820      NaN
567        0.12400      NaN
568        0.07039      NaN

```

```
[569 rows x 33 columns]
```

The output below shows that there are 569 observations and 33 columns. The diagnosis variable is stored as an object, while the majority of the predictor variables are numerical. Importantly, all of the variables containing measurements have 569 non-null observations, indicating that there are no missing values in these variables.

```
[2]: #Assessing if there are any null values and the size of the data set
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                    569 non-null    int64
1   diagnosis                            569 non-null    object
2   radius_mean                          569 non-null    float64
3   texture_mean                         569 non-null    float64
4   perimeter_mean                       569 non-null    float64
5   area_mean                            569 non-null    float64
6   smoothness_mean                      569 non-null    float64
7   compactness_mean                     569 non-null    float64
8   concavity_mean                       569 non-null    float64
9   concave points_mean                  569 non-null    float64
10  symmetry_mean                        569 non-null    float64
11  fractal_dimension_mean                569 non-null    float64
12  radius_se                             569 non-null    float64
13  texture_se                            569 non-null    float64
14  perimeter_se                          569 non-null    float64
15  area_se                              569 non-null    float64
16  smoothness_se                        569 non-null    float64
17  compactness_se                       569 non-null    float64
18  concavity_se                         569 non-null    float64
19  concave points_se                    569 non-null    float64
20  symmetry_se                          569 non-null    float64
21  fractal_dimension_se                  569 non-null    float64
22  radius_worst                         569 non-null    float64
23  texture_worst                        569 non-null    float64
24  perimeter_worst                      569 non-null    float64

```

```

25  area_worst                569 non-null    float64
26  smoothness_worst         569 non-null    float64
27  compactness_worst        569 non-null    float64
28  concavity_worst          569 non-null    float64
29  concave points_worst     569 non-null    float64
30  symmetry_worst           569 non-null    float64
31  fractal_dimension_worst  569 non-null    float64
32  Unnamed: 32              0 non-null    float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB

```

The Unnamed: 32 variable contains no observations and does not provide any information for the analysis. The id variable is also not a measurement of a tumour characteristic; it is an identifier for each observation. Neither variable is therefore useful as a predictor of diagnosis.

```

[3]: #Dropping 2 irrelevant columns from the data frame
df.drop(['id', 'Unnamed: 32'], axis = 1, inplace = True)

```

The predictor variables are divided into three groups according to how the measurements were calculated. The first group contains the mean characteristics, the second contains the standard error characteristics, and the third contains the worst characteristics. Separating the variables allows the predictive performance of each type of measurement to be investigated independently. Rather than building one decision tree containing all available variables, three separate models can be compared under the same modelling approach. Each data frame keeps diagnosis as the target variable.

```

[5]: #Defining a new data set that looks at the mean of characteristics
df_mean = df[['diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
              'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
              'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean']]

#Defining a new data set that looks at the standard error characteristics
df_se = df[['diagnosis', 'radius_se', 'texture_se', 'perimeter_se', 'area_se',
             'smoothness_se',
             'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
             'fractal_dimension_se']]

#Defining a new data set that looks at the worst of the characteristics
df_worst = df[['diagnosis', 'radius_worst', 'texture_worst',
               'perimeter_worst', 'area_worst', 'smoothness_worst',
               'compactness_worst', 'concavity_worst', 'concave points_worst',
               'symmetry_worst', 'fractal_dimension_worst']]

```

### 1.3 Decision trees

A decision tree is a machine learning algorithm that can be used for classification. In this analysis, the target variable is diagnosis, which identifies whether a tumour is malignant (M) or benign (B).

Three separate decision trees are constructed in this analysis. The first uses the mean characteris-

tics, the second uses the standard error characteristics, and the third uses the worst characteristics. Their performance can then be compared to determine which group of measurements provides the most useful information for distinguishing between tumours.

To assess the quality of each model, 5-fold cross-validation is used i.e.e the data is divided into five folds. The model is trained using four folds and evaluated using the remaining fold. This process is repeated five times so that each fold is used as the validation set once.

The five accuracy values outputted in each section therefore represent the model's performance on five different subsets of the data. The mean accuracy provides an overall measure of predictive performance, while the standard deviation indicates how much the model's performance varies between folds.

The code used in this section is taken from the websites linked below. Variables have been altered to match this data set. <https://www.datacamp.com/tutorial/decision-tree-classification-python>  
[https://scikit-learn.org/stable/modules/cross\\_validation.html](https://scikit-learn.org/stable/modules/cross_validation.html)

### 1.3.1 Decision tree for mean characteristics

The first decision tree uses the ten mean measurements as features. These include the mean radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension.

```
[119]: # Load sklearn libraries
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

[129]: # Split dataset in features and target variable
features_mean = ['radius_mean', 'texture_mean', 'perimeter_mean',
                 'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
                 'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean']

# Features
X = df_mean[features_mean]

# Target variable is the diagnosis
y = df_mean.diagnosis

# Create Decision Tree classifier object
clf_mean = DecisionTreeClassifier()

# 5-fold cross-validation
# scoring='accuracy' calculates correct predictions / total predictions
scores = cross_val_score(clf_mean, X, y, cv=5, scoring='accuracy')

# Display the results
print("Accuracy scores for each fold:", scores)
print(f"Mean Accuracy: {scores.mean():.2f}")
print(f"Standard Deviation: {scores.std():.2f}")
```

Accuracy scores for each fold: [0.90350877 0.9122807 0.9122807 0.94736842 0.92920354]

Mean Accuracy: 0.92

Standard Deviation: 0.02

The decision tree correctly classifies approximately 92% of observations on average across the five validation folds. The standard deviation is 0.02, indicating relatively little variation in performance between the folds. This suggests that the mean characteristics contain substantial information about whether a tumour is malignant or benign.

### 1.3.2 Decision tree for standard error of characteristics

The second decision tree is constructed using the standard error measurements. These variables describe the variability associated with the corresponding characteristics.

```
[130]: # Split dataset in features and target variable
features_se = ['radius_se', 'texture_se', 'perimeter_se', 'area_se', '
    'smoothness_se',
               'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
               'fractal_dimension_se']

# Features
X = df_se[features_se]

# Target variable is the diagnosis
y = df_se.diagnosis

# Create Decision Tree classifier object
clf_se = DecisionTreeClassifier()

# 5-fold cross-validation
# scoring='accuracy' calculates correct predictions / total predictions
scores = cross_val_score(clf_se, X, y, cv=5, scoring='accuracy')

# Display the results
print("Accuracy scores for each fold:", scores)
print(f"Mean Accuracy: {scores.mean():.2f}")
print(f"Standard Deviation: {scores.std():.2f}")
```

Accuracy scores for each fold: [0.79824561 0.86842105 0.86842105 0.8245614 0.81415929]

Mean Accuracy: 0.83

Standard Deviation: 0.03

The decision tree correctly classifies approximately 83% of observations on average across the five validation folds and the standard deviation is 0.03. The results suggest that the standard error measurements are less effective predictors of tumour diagnosis than the mean measurements. The slightly larger standard deviation also indicates more variation in predictive performance between the folds.

### 1.3.3 Decision tree for worst of characteristics

The final decision tree uses the ten worst measurements. These features represent the corresponding characteristics of the cell nuclei using the mean of the three largest values present in the breast mass image.

```
[134]: # Split dataset in features and target variable
features_worst = ['radius_worst', 'texture_worst',
                  'perimeter_worst', 'area_worst', 'smoothness_worst',
                  'compactness_worst', 'concavity_worst', 'concave points_worst',
                  'symmetry_worst', 'fractal_dimension_worst']

# Features
X = df_worst[features_worst]

# Target variable is the diagnosis
y = df_worst.diagnosis

# Create Decision Tree classifier object
clf_worst = DecisionTreeClassifier()

# 5-fold cross-validation
# scoring='accuracy' calculates correct predictions / total predictions
scores = cross_val_score(clf_worst, X, y, cv=5, scoring='accuracy')

# Display the results
print("Accuracy scores for each fold:", scores)
print(f"Mean Accuracy: {scores.mean():.2f}")
print(f"Standard Deviation: {scores.std():.2f}")
```

```
Accuracy scores for each fold: [0.92982456 0.92105263 0.95614035 0.94736842
0.90265487]
Mean Accuracy: 0.93
Standard Deviation: 0.02
```

The decision tree correctly classifies approximately 93% of observations on average across the five validation folds and the standard deviation is 0.02. This is the highest mean accuracy of the three models. The results indicate that the worst characteristics provide slightly more useful information for distinguishing between tumours than the mean characteristics.

## 1.4 Visualising the decision trees

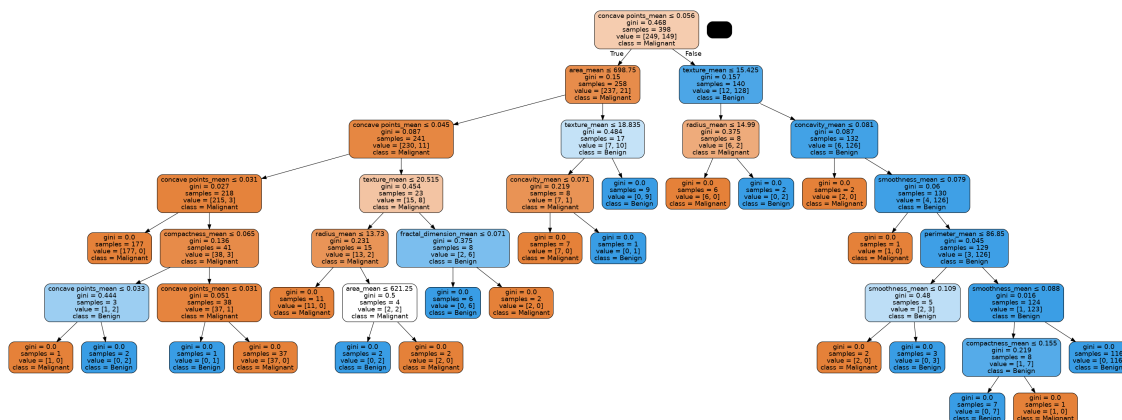
The advantage of decision trees is that they are easy to visualise because the classification process can be represented graphically. Visualising the trees also allows the individual decision rules used by the classifier to be seen.

Each node represents a decision based on one of the predictor variables. The value at which the data are split determines which observations follow each branch. The process continues until the observations reach a leaf node, where the model assigns a predicted class.

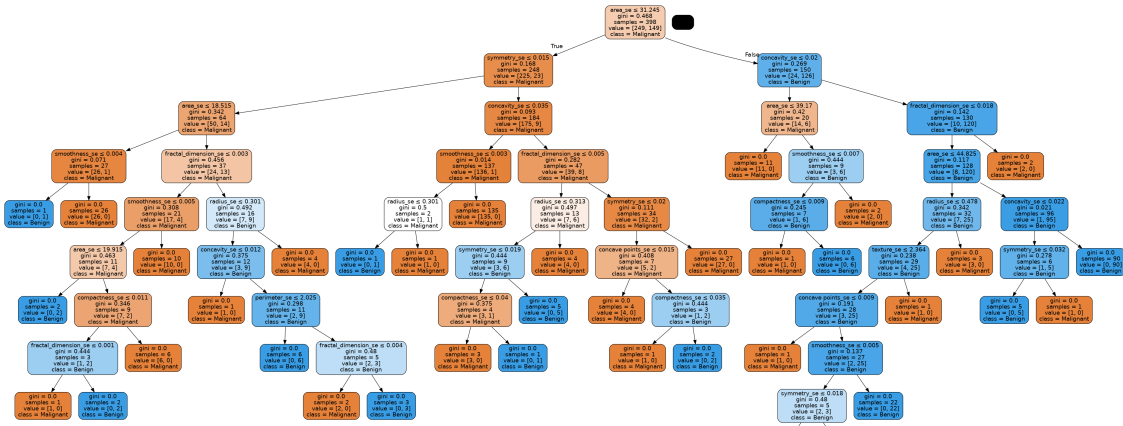
```
#Displaying the decision tree for the mean characteristics
from sklearn.tree import export_graphviz
from io import StringIO
from IPython.display import Image
import pydotplus

class_names = ['Malignant', 'Benign']
dot_data = StringIO()
export_graphviz(clf_mean, out_file=dot_data,
                filled=True, rounded=True,
                special_characters=True, feature_names=features_mean,
                class_names=class_names) # Assuming class_names variable exists
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
graph.write_png('breastcancer_mean.png')
Image(graph.create_png())

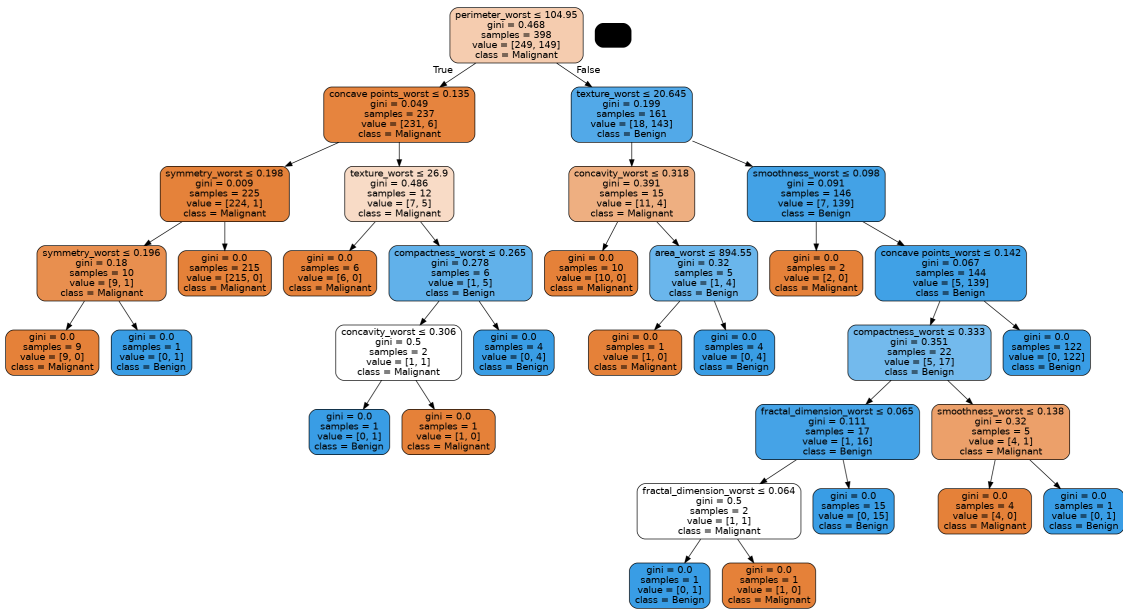
class_names = ['Malignant', 'Benign']
dot_data = StringIO()
export_graphviz(clf_se, out_file=dot_data,
                filled=True, rounded=True,
                special_characters=True, feature_names=features_se,
                class_names=class_names) # Assuming class_names variable exists
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
graph.write_png('breastcancer_mean.png')
Image(graph.create_png())
```



### 1.4.2 Decision tree of standard error characteristics



### 1.4.3 Decision tree of worst characteristics



## 1.5 Conclusion

This analysis investigated whether different groups of breast tumour characteristics could be used to predict whether a tumour is malignant or benign using decision tree classification. The results showed that the model based on the worst characteristics achieved the highest mean accuracy of 93%, followed by the model using the mean characteristics with an accuracy of 92%. The model using the standard error characteristics performed less well, achieving a mean accuracy of 83%. The relatively small standard deviations for the mean and worst models indicate that their performance was consistent across the five cross-validation folds.

Overall, the results suggest that the mean and particularly the worst measurements contain more useful information for distinguishing between malignant and benign tumours than the standard error measurements in this analysis.